

IT Support System

Node configuration, credentials, and API key setup guide

1. Supabase Credentials

1.1 Project URL & keys

1. Supabase Dashboard → Settings → API.
2. Copy `Project URL` and the `anon` key → frontend `.env.local`.
3. Copy the `service_role` key → n8n Config node (`SUPABASE_SERVICE_ROLE_KEY`). Never expose this key to the frontend.

1.2 Database Webhook (triggers the whole automation)

1. Sidebar → **Integrations** → **Database Webhooks** (not the Database → Triggers page — that only calls Postgres functions).
2. Click **Create a new hook**.
3. Name: `ticket_created_webhook`.
4. Table: `tickets`. Events: **Insert** only.
5. Type of webhook: **HTTP Request**.
6. Method: `POST`. URL: `https://<your-n8n-host>/webhook/ticket-created`.
7. Save.

SQL alternative — same effect, run in the SQL editor:

```
create trigger ticket_created_webhook
after insert on public.tickets
for each row
execute function supabase_functions.http_request(
  'https://<your-n8n-host>/webhook/ticket-created',
  'POST', '{"Content-Type":"application/json"}', '{}', '5000'
);
```

If tunneling n8n with ngrok, the free-tier URL changes on every restart — update this webhook whenever it does.

2. n8n Credentials

2.1 Jira Cloud API

1. n8n → Credentials → New → **Jira SW Cloud API**.
2. **Email**: your Atlassian account email.
3. **API Token**: generate at `id.atlassian.com/manage-profile/security/api-tokens` → Create API token → copy immediately (shown once).
4. **Domain**: your Jira site URL as seen in the browser, e.g. `https://yourcompany.atlassian.net`.
5. Save.

2.2 Gmail OAuth2

n8n Cloud: New credential → Gmail OAuth2 API → Connect my account → sign in → grant access. Done.

Self-hosted (custom Google Cloud OAuth client):

1. Google Cloud Console → create/select a project.
2. APIs & Services → Library → search **Gmail API** → **Enable**. (Most common cause of a "Forbidden" send error is forgetting this step.)
3. APIs & Services → OAuth consent screen → User type External → fill app name/support/dev email.
4. Add scope `https://www.googleapis.com/auth/gmail.send`.
5. Add your sending Gmail address under **Test users** (required while app status is "Testing").
6. APIs & Services → Credentials → Create Credentials → OAuth client ID → Application type **Web application**.
7. Authorized redirect URI: copy the exact value n8n shows on its Gmail credential screen.
8. Paste the generated Client ID / Client Secret into n8n's Gmail OAuth2 credential.
9. Click **Connect my account** in n8n, sign in, grant access, Save.

2.3 Telegram bot token

1. Message **@BotFather** on Telegram → `/newbot` → follow prompts → copy the bot token.
2. n8n → Credentials → New → Telegram API → paste the Access Token → Save (should show "Connection tested successfully").
3. Create/choose a Telegram group for ops alerts → add the bot to it → send any message in the group.
4. In a browser, open: `https://api.telegram.org/bot<TOKEN>/getUpdates`.
5. Find `"chat":{"id": -100xxxxxxxxxx, ...}` — that negative number is the group Chat ID, needed in both Config nodes as `TELEGRAM_SUPPORT_GROUP_CHAT_ID`.

3. Gemini API Key

1. Go to `aistudio.google.com/apikey`.
2. Create an API key (free tier, no card required for low volume).
3. Paste into the Config node's `GEMINI_API_KEY`.

4. Config Node values (both workflows)

Field	Where to find it
<code>SUPABASE_URL</code>	Supabase → Settings → API → Project URL
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Supabase → Settings → API → service_role key
<code>SUPABASE_STORAGE_BUCKET</code>	<code>ticket-attachments</code> (created by schema.sql)
<code>GEMINI_API_KEY</code>	Section 3 above
<code>JIRA_PROJECT_ID</code> , <code>JIRA_ISSUETYPE_BUG_ID</code> , <code>JIRA_ISSUETYPE_TASK_ID</code>	Jira project settings, or click "By ID" mode on the Create Jira Issue node's Project/Issue Type fields to browse IDs
<code>TELEGRAM_SUPPORT_GROUP_CHAT_ID</code>	Section 2.3 above

TEAM_EMAIL_*	Your team mailboxes (Service Desk, Network, Security, Infrastructure, App Support)
--------------	--

Keep the error-handler workflow's smaller Config node (SUPABASE_URL , SUPABASE_SERVICE_ROLE_KEY , TELEGRAM_SUPPORT_GROUP_CHAT_ID) in sync with the main one.

5. Node-by-node configuration (main workflow)

5.1 Webhook: Ticket Created

Path must exactly match the Supabase webhook URL, e.g. ticket-created . No further config.

5.2 Config

Code node — paste in the values from Section 4 as a plain JS object.

5.3 Fetch Full Ticket / Update Ticket: AI Fields / Update Ticket: Report URL

HTTP Request nodes hitting Supabase REST

({{ \$('Config').first().json.SUPABASE_URL }}/rest/v1/tickets). Auth via apikey + Authorization: Bearer headers using the service role key. Query param id=eq.{{ \$json.id }} selects the row.

5.4 AI Analysis & Business Rules

Code node. Calls Gemini, applies HIGH_PRIORITIES / FORCE_JIRA_CATEGORIES rules to set jira_required . Edit these arrays directly in the node to change triage logic.

5.5 IF: Jira Required?

Condition: {{ \$json.jira_required }} is true .

5.6 Create Jira Issue

- **Project:** By ID → {{ \$('Config').first().json.JIRA_PROJECT_ID }}
- **Issue Type:** By ID → {{ \$json.ai_severity === 'Critical' ? \$('Config').first().json.JIRA_ISSUETYPE_BUG_ID : \$('Config').first().json.JIRA_ISSUETYPE_TASK_ID }}
- **Summary:** [{{ \$json.ticket_code }}] {{ \$json.subject }}
- **Description:** multi-line, includes AI summary + original description.
- **Priority:** omit this field entirely unless your Jira project's issue screen has Priority enabled — otherwise Jira returns a 400 "priority selected is invalid" error even for correct-looking names.
- **Labels:** omit this field entirely — the n8n Jira node commonly serializes an array expression as a plain string here, which Jira's API rejects with "Specify the value for Labels in an array". Category/severity are already preserved in the Description.

5.7 Save Jira Key to Supabase

PATCH to tickets , body {"jira_issue_key": \$json.key} , matched by id .

5.8 Merge Branches

Mode: Append (not "Choose Branch" with "Wait for All Inputs to Arrive" — since the IF node only ever sends data down one of its two outputs per execution, "wait for all" will just hang and output nothing).
Number of Inputs: 2.

5.9 Build & Upload Report (.txt)

Code node. Formats the plain-text incident report, uploads to Supabase Storage under `reports/`, resolves the recommended team's mailbox from Config into a `team_email` field on the item.

5.10 Notification fan-out

Important: connect *all five* of the following nodes directly off **Build & Upload Report (.txt)**, in parallel — do not chain them one after another. Any node downstream of an HTTP Request (Supabase update) or a Gmail/Telegram "send" response loses the original ticket JSON and the binary file, causing "undefined" recipients or "no binary file found" errors.

- **Update Ticket: Report URL** — PATCH Supabase, body `{"pdf_url": $json.report_url, "status": "In Progress"}` (confirm your actual column name in the `tickets` table before setting this key).
- **Send Email to Requester** — To: `{{ $json.email }}`, HTML message, attachment field name `data`.
- **Send Email to Support Team** — To: `{{ $json.team_email }}`, same attachment setup. Requires the Gmail API to be enabled in Google Cloud (Section 2.2) or you'll get a Forbidden error even with a valid credential attached.
- **Send Telegram Summary** — Chat ID: `{{ $('Config').first().json.TELEGRAM_SUPPORT_GROUP_CHAT_ID }}`, formatted Markdown text.
- **Send Telegram Report** — same Chat ID, Operation: Send Document, Input Binary Field: `data` — must also connect directly off Build & Upload Report, not after Send Telegram Summary, for the same binary-loss reason above.

5.11 Workflow Settings → Error Workflow

Settings (gear icon) → Error Workflow → select the imported **error-handler workflow**. Both workflows must be Active for this and the webhook trigger to work.

6. Quick verification checklist

- ☐ Supabase webhook fires on ticket insert (check n8n Executions tab)
- ☐ Config node values filled in both workflows
- ☐ Jira, Gmail, Telegram credentials all show a successful test/connection
- ☐ Gmail API enabled in Google Cloud (self-hosted only)
- ☐ Merge Branches set to Append
- ☐ Priority and Labels removed from Create Jira Issue (unless your project supports them)
- ☐ All 5 notification/update nodes connected directly off Build & Upload Report (.txt)
- ☐ Both workflows Active, errorWorkflow linked
- ☐ End-to-end test ticket submitted from the frontend and traced through Executions